Modèles algorithmiques fondamentaux sur structures à accès direct

Thème principal : Initiation à l'algorithmique et patrons de conception (Design Patterns) sur structures à accès direct

Synthèse exécutive : Ce cours magistral pose les fondements méthodologiques indispensables de l'algorithmique en BUT 1 Informatique. Avant toute écriture de code, l'informaticien doit analyser le contrat d'interface et le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois grands modèles algorithmiques canoniques réutilisables sur des structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence. Une rigueur absolue est exigée sur le respect des bornes d'indices (de 0 à N-1) pour éviter les débordements de mémoire ('out of bounds'), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer une double condition d'arrêt pour tout algorithme de recherche.

1. Typologie et Mécanismes d'Accès aux Collections de Données

Avant de concevoir le moindre algorithme, l'étape zéro de l'informaticien consiste à analyser le contrat d'interface et la nature de la collection de données manipulée. Le choix d'un modèle algorithmique dépend strictement des opérations permises par la structure. Si l'on travaille sur une structure à accès direct (tableaux, chaînes de caractères), on peut cibler n'importe quel indice immédiatement. En revanche, sur un flux séquentiel (fichier texte brut, flux CSV lu ligne par ligne), il est impossible de sauter directement à un indice donné : il faut obligatoirement parcourir les éléments un à un depuis le début. Tenter d'utiliser une notation par indice (ex: T[i]) sur un flux séquentiel constitue une erreur conceptuelle critique qui invalide totalement l'algorithme.

- Structure de données : Collection organisée regroupant des éléments de même type (tableaux, chaînes de caractères).
- Accès direct : Capacité d'atteindre instantanément n'importe quel élément via un indice ou une clé (ex: T[i]) en temps constant O(1), sans parcourir les précédents.
- Accès séquentiel : Contrainte imposant de se positionner au début puis de progresser pas à pas (début, suivant, fin) sans saut direct possible (ex: fichiers CSV bruts, listes chaînées simples).
- Accès dynamique : Structure hybride proposant simultanément des primitives d'accès séquentiel et d'indexation directe.
- Incompatibilité structurelle : Interdiction absolue d'appliquer un modèle nécessitant l'accès direct (ex: T[i]) sur un flux purement séquentiel.

Cas concret enseignant : *Le professeur met en garde contre l'erreur classique observée en contrôle : tenter d'écrire 'T[i]' sur un fichier CSV ou un fichier texte ouvert. Comme le système d'exploitation ne fournit qu'un curseur séquentiel, un tel code est sémantiquement faux et conduit directement à un zéro sur la question.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle algorithmique s'applique dès lors qu'aucune exclusion n'est permise entre deux bornes définies. Il répond à des problématiques telles que : afficher toutes les valeurs d'un tableau, initialiser chaque case à zéro, ou convertir chaque caractère d'une chaîne en majuscule. En pseudo-code normalisé, pour un tableau de taille N, l'indice i démarre à 0 et termine impérativement à N - 1. Le traitement est appliqué aveuglément à chaque tour de boucle sans test préalable.

- Objectif : Appliquer un traitement uniforme et inconditionnel à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : Utilisation obligatoire d'une boucle bornée 'Pour' (car le nombre d'itérations est calculable à l'avance : Fin - Début + 1).

• Gestion stricte des bornes : Pour une structure de N éléments indicée à partir de 0, la boucle varie strictement de 0 à N - 1.

• Erreur critique (Out of Bounds) : Faire varier l'indice jusqu'à N provoque un dépassement de capacité mémoire.

Cas concret enseignant : *Exemple concret donné en amphi : 'Afficher un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente des débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat à l'exécution lors de la tentative de lecture de la case mémoire T[N] qui n'appartient pas au tableau.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Le modèle du parcours complet conditionné conserve l'inspection systématique de l'ensemble des éléments, mais protège l'exécution de l'action de traitement par une condition logique. Il est utilisé pour compter des occurrences particulières (ex: compter les voyelles dans un texte), filtrer des données (ex: afficher uniquement les entiers pairs) ou accumuler des valeurs spécifiques. Si la condition implique des éléments adjacents (ex: T[i] > T[i-1]), le concepteur doit impérativement réajuster les bornes de la boucle (ex: démarrer à 1 au lieu de 0) pour empêcher tout déréférencement hors limites.

• Parcours exhaustif : Tous les éléments de la zone sont inspectés de la borne de début à la borne de fin.

• Traitement sélectif : L'action n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).

• Architecture : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.

• Conditions complexes : Le prédicat peut combiner plusieurs critères logiques (ET, OU) ou comparer l'élément courant avec ses voisins immédiats (i-1 ou i+1).

Cas concret enseignant : *L'analogie de la chaîne de montage en usine : 'Chaque pièce défile obligatoirement devant le contrôleur qualité (parcours complet systématique d'inspection), mais le contrôleur n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).'*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux deux premiers modèles, la recherche de première occurrence stoppe l'itération dès que la cible est identifiée. L'informaticien doit adopter une démarche pessimiste en postulant que l'élément n'existe pas. Un algorithme de recherche valide DOIT posséder deux conditions d'arrêt : (1) le succès de la recherche, (2) l'épuisement complet de la collection. Deux variantes structurelles existent : la variante 1 (rupture immédiate, idéale pour récupérer la position exacte) et la variante 2 (Tant que global, plus simple à structurer syntaxiquement).

• Pessimisme algorithmique : Initialisation systématique d'un indicateur booléen 'trouvé' à Faux avant d'entamer l'exploration.

• Itération non bornée : Utilisation obligatoire d'une boucle 'Tant que' car le nombre de répétitions est inconnu a priori.

• Double condition d'arrêt obligatoire : Arrêt dès que l'élément est trouvé OU dès que la structure a été entièrement explorée sans succès.

• Variante 1 (Rupture immédiate) : Interruption nette dès la découverte ; conserve l'indice i exactement positionné sur l'élément trouvé.

• Variante 2 (Tant que global) : Test unique en tête ('Tant que i < N et NON trouvé') ; syntaxiquement plus standard mais avance d'un pas supplémentaire avant de sortir.

Cas concret enseignant : *Devise du prof : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans votre collection, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle que tout algorithme de recherche présenté en examen avec une seule condition d'arrêt est sanctionné par une note éliminatoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Comme une structure à accès dynamique propose à la fois des primitives d'accès séquentiel et d'accès direct, vous pouvez tout à fait exploiter ses mécanismes d'accès direct pour y appliquer un parcours systématique ou conditionné.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que le choix dépend de votre objectif : la variante 1 (avec rupture) est idéale si vous avez besoin de récupérer la position exacte (l'indice) de l'élément trouvé car elle s'arrête net sur la case. La variante 2 (Tant que global) offre une syntaxe plus épurée et standard, mais nécessite un réajustement d'indice (i - 1) si l'on souhaite exploiter la position.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : C'est une adaptation tout à fait classique, mais vous devez impérativement ajuster les bornes de votre boucle (par exemple démarrer à l'indice 1 au lieu de 0 si vous comparez avec l'élément i-1) pour éviter un plantage critique de dépassement mémoire (out of bounds sur T[-1]).

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur (par exemple trouver toutes les occurrences du nombre 5 dans un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser une recherche de première occurrence (qui s'arrête au premier trouvé), mais un 'Parcours complet avec traitement conditionnel' qui va inspecter l'intégralité du tableau de 0 à N-1 et enregistrer ou afficher chaque indice où la condition est vérifiée.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. On doit inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle canonique est donc le 'Parcours complet avec traitement conditionnel'. Étape 2 : Détermination des bornes. En indexation standard (commençant à 0), pour N éléments, les indices s'étendent de 0 à N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Structure de contrôle itérative. Nombre d'itérations connu -> Boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Filtre logique et action. À chaque tour, évaluer 'Si T[i] > 0 Alors' et exécuter 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit le parcours complet exhaustif sans risque de débordement d'indice (arrêt strictement à N-1) et filtre sélectivement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence (car il s'agit de tester l'existence d'une valeur et de s'interrompre dès qu'on la trouve).

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Parce qu'en indexation standard à partir de 0, les indices d'un tableau de N éléments s'étendent de 0 à N - 1. Tenter d'accéder à l'indice N provoque une erreur critique de dépassement de capacité (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure de contrôle d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt minimales : (1) Le succès de la recherche (élément trouvé), et (2) L'échec par épuisement complet de la collection (fin de structure atteinte).